



Técnicas de los Sistemas Inteligentes

Enunciado de la Práctica 2

En esta práctica, el alumnado debe resolver siete problemas usando MiniZinc. Cada ejercicio se resolverá en un **fichero MZN por separado (siguiendo estrictamente la nomenclatura indicada)**. La solución entregada debe ser **correcta y completa**, es decir, todas las soluciones válidas deben ser admitidas por la codificación entregada y ésta no puede admitir ninguna solución no válida. **El código debe estar perfectamente comentado, explicando y justificando la solución presentada, variables y restricciones usadas para codificar cada problema, y cualquier otro aspecto relativo a las decisiones tomadas durante el proceso de codificación. Un código cuyos comentarios sean, a criterio del profesor, deficientes podrá invalidar total o parcialmente aquellos problemas a los que afecte.** Para la resolución de todos los problemas se debe usar el **solver Gecode**, solver pre-instalado en el IDE de Minizinc, con sus valores por defecto. Los problemas con **errores sintácticos** en MiniZinc automáticamente califican con una puntuación de **0 (cero) puntos**. Por otro lado, **no se permite solicitar información de entrada al usuario**: las distintas opciones o valores a probar se deben incluir en el fichero, dejando comentados todos los valores de entrada solicitados menos uno. Debe estar muy claro qué debe comentar/descomentar el profesor para poder ejecutar el código.

Como ya se indicó en la Práctica 1, no supone ningún problema utilizar ChatGPT u otras herramientas de IA generativa como apoyo para el desarrollo de las prácticas. De hecho, incluso se recomienda su uso, dado que se trata, sin duda, de herramientas de gran utilidad en la realización de múltiples tareas, y que nos permiten aumentar nuestra productividad y, bien empleadas, enriquecer nuestro proceso de aprendizaje en gran medida. No obstante, cuando se empleen, se debe indicar con claridad y especificar de qué modo y en qué partes se han usado (y por qué). Lo contrario, puede suponer una penalización severa en la calificación recibida (incluyendo el suspenso de la práctica y/o de la asignatura). Véase <https://ceprud.ugr.es/formacion-tic/inteligencia-artificial> (especialmente, la sección titulada “Recomendaciones e ideas para estudiantes”). También, al igual que en la práctica anterior, es bastante posible que el profesorado evalúe un porcentaje determinado de entregas por medio de un examen oral.

Junto con lo anterior, se entregará una **memoria en formato PDF (con un tamaño máximo de 14 páginas¹)** en donde se indicarán y analizarán los resultados obtenidos, así como se proporcionará una explicación clara y detallada de la codificación empleada. En la memoria también se debe incluir la respuesta a las preguntas planteadas en cada ejercicio (no se incluirá código; éste ya se entrega en el fichero MZN).

Cada problema tiene una puntuación que se indica en el propio título del ejercicio, y dicha puntuación incluye tanto la codificación MZN como la respuesta razonada en la memoria. La

¹ Estas 14 páginas incluyen todo. Es decir, si se quiere incluir una portada, un índice o una sección bibliográfica, esas páginas se contabilizarían dentro del máximo de 14.



ugr

Universidad de Granada
Departamento de Ciencias de la Computación
e Inteligencia Artificial



entrega consistirá en un **fichero ZIP** que contenga la memoria de la práctica (PDF) y los ficheros MZN de cada problema.

Fecha de entrega: 12/05/2026 23:59

Modo de entrega: A través de la plataforma **PRADO**, en la tarea creada para ello.



1. Problema de la fórmula de una bebida isotónica (1.5 pts)

Una empresa de nutrición deportiva desea formular una bebida isotónica mezclando varios ingredientes líquidos. Se dispone de 6 ingredientes, cada uno con un coste y una composición nutricional por cada 100 ml:

Ingrediente	Coste (€)	Azúcar (g)	Sodio (mg)	Cafeína (mg)
Agua base	0.02	0	0	0
Jarabe A	0.12	18	40	0
Jarabe B	0.10	12	20	0
Concentrado salino	0.08	0	180	0
Extracto energético	0.20	4	10	30
Aroma cítrico	0.06	2	5	0

Se desea obtener una cantidad total determinada (denominada “Volumen total buscado (en ml)” en la tabla de abajo) de bebida isotónica. Dicha cantidad será un parámetro de entrada del modelo. Cada cantidad de ingrediente debe expresarse en múltiplos de 10 ml. La bebida solicitada debe cumplir ciertas restricciones nutricionales proporcionales al volumen total. En particular, se proporcionan los siguientes valores de referencia por cada 1000 ml de bebida, que deben respetarse proporcionalmente al volumen total buscado:

- entre 70 y 85 g de azúcar,
- entre 400 y 475 mg de sodio,
- entre 60 y 90 mg de cafeína.

Además:

- El aroma cítrico debe usarse obligatoriamente en entre el 3% y el 20% del volumen total.
- El extracto energético no puede superar el 40% del volumen total.
- Debe utilizarse exactamente uno de los dos jarabes (Jarabe A o Jarabe B).
- Si se usa Jarabe A, debe emplearse al menos en el 5% del volumen total.
- Si se usa concentrado salino, debe emplearse al menos en el 3% del volumen total.
- Como mucho pueden usarse 4 ingredientes distintos.

- a. Codificar una solución en MiniZinc usando exclusivamente codificación entera para calcular cualquier solución válida. Dicho fichero se denominará Ejercicio1a.mzn. No puede definirse en todo el fichero ninguna variable float. La salida debe mostrar:

- los ml usados de cada ingrediente,
- el coste total,
- la cantidad total de azúcar,
- la cantidad total de sodio,
- la cantidad total de cafeína.

Ejemplo de formateo de salida que se debe emplear para mostrar la composición de una solución encontrada:



Cantidades usadas:

Agua base: 0 ml
Jarabe A: 0 ml
Jarabe B: 3140 ml
Concentrado salino: 710 ml
Extracto energetico: 1000 ml
Aroma citrico: 150 ml

Coste total = 579.8 centimos
Azucar total = 419.8 g
Sodio total = 2013.5 mg
Cafeina total = 300 mg

A continuación, una vez haya finalizado su implementación, complete la siguiente tabla:

Volumen total buscado (en ml)	Primera solución encontrada	Número total de soluciones	<i>Runtime</i> (en milisegundos) para calcular todas las posibles soluciones
200			
1000			
5000			
10000			

Nota1: Para calcular todas las soluciones (en lugar de una sola), se debe desmarcar la casilla “*Stop after this many solutions (uncheck for all)*” en el menú “*Show configurator editor*” (“*User defined behavior*”). Igualmente, para imprimir el número total de soluciones, debe activarse la opción “*Output solving statistics*”, también en este menú. El número de soluciones aparece en la consola en el campo “%%mzn-stat: nSolutions” (cuando se ejecuta con el solver Gecode).

Nota2: Para el *runtime*, se recomienda usar el valor reportado por MiniZinc en la última línea “*Finished in ...*” (ya que el valor reportado en las estadísticas como “*solveTime*” no incluye todo el proceso de resolución y, a veces, es algo más inestable por la forma en que es medido).

- b. Modifique el código del apartado anterior para que el coste total sea mínimo. Este fichero se debe denominar Ejercicio1b.mzn. A continuación, se debe generar una nueva tabla de resultados con una columna de “solución óptima”, donde se indique la solución óptima encontrada para los volúmenes solicitados, y otra columna de *runtime*.

Volumen total buscado (en ml)	Solución óptima	Runtime (en milisegundos)
200		
1000		



5000		
10000		

c. Responda razonadamente a las siguientes cuestiones:

- A medida que vamos aumentando el volumen en el apartado (a), ¿qué ocurre? ¿Cómo escala (a nivel de tiempos de resolución) nuestra codificación con volúmenes progresivamente más elevados? Por ejemplo, ¿cuánto se tardaría en encontrar todas las soluciones para un volumen de 1 millón de ml?
- A priori, intuitivamente, ¿qué debería ser más rápido: resolver un CSP (como el del apartado (a)) o un COP (como el del apartado (b))? En otras palabras, si, en lugar de aplicar “solve satisfy;”, intentásemos minimizar una métrica, ¿el problema sería más fácil o difícil de resolver? ¿Deberíamos tardar más o menos tiempo en encontrar una solución? ¿Por qué? En el caso de este ejercicio concreto, ¿esta intuición se cumple? Si la respuesta es negativa, ¿a qué puede deberse?
- La codificación de un problema resulta esencial para su correcta resolución. En este ejercicio, por ejemplo, hemos forzado una codificación entera y, muy probablemente, una codificación flotante también sería válida. En cambio, hay otras problemáticas en donde una codificación flotante podría resultar inadecuada. Veamos, por ejemplo, el siguiente código MiniZinc, en donde, dado un importe y una serie de monedas, se busca la combinación de monedas que equivale al importe inicial. ¿Qué ocurre y a qué se puede deber? En caso de percibir algún problema, ¿cómo se podría resolver?

```
include "globals.mzn";
float: importe = 1.47;
% numero de monedas diferentes
int: n = 9;
% valor de las monedas
array[1..n] of float: valor = [0.01, 0.03, 0.05, 0.10, 0.25, 0.50, 1.0, 2.0, 5.0];
% array para la cantidad de monedas de cada valor
array[1..n] of var 0.0..importe: monedas;
% numero de monedas usadas
var float: nmonedas = sum(i in 1..n) (monedas[i]);
% La suma debe ser la cantidad buscada
constraint sum(i in 1..n)(monedas[i]*valor[i]) == importe;
solve satisfy;
output [
  "monedas: " ++ show(monedas) ++ "\n" ++
  "total monedas: " ++ show(nmonedas)
];
```



2. Puzzle lógico (1 pto)

En una feria tecnológica organizada por la ETSIT, 6 estudiantes de TSI presentan sus proyectos. Cada estudiante presenta un proyecto distinto, en un aula distinta y en una hora distinta. Los estudiantes se llaman Alba, Bruno, Carla, Diego, Elena, y Pablo; y los proyectos se titulan “A* cuántico” (proyecto 1), “Demostración matemática de que RTA* es un algoritmo absurdo” (proyecto 2), “Modelado de la incertidumbre a la hora de aprobar TSI sin estudiar nada” (proyecto 3), “Poda alfa-beta aplicada al encaje de bolillos” (proyecto 4), “Planificación automática jerárquica híbrida dedicada a la yuxtaposición de los mundos paralelos” (proyecto 5), y “Brazo robótico para jugar a la pocha” (proyecto 6). Las aulas en las que se presentan los proyectos son la A1, A2, B1, B2, C1, y C2; mientras que las posibles horas de presentación son 15:00, 16:00, 17:00, 18:00, 19:00 y 20:00. A partir de las siguientes pistas, determine qué proyecto presenta cada estudiante, en qué aula y a qué hora:

- Bruno es el primero en presentar.
- El proyecto 4 se presenta en el aula B2 y antes que el proyecto 1.
- El proyecto 1 se presenta inmediatamente antes que el proyecto 3, en un aula que no es ni la B2 ni la A1.
- Diego presenta exactamente una hora después que Alba.
- Carla presenta exactamente una hora después que Diego.
- Elena presenta en el aula C1, y su presentación tiene lugar después de Carla pero antes de Pablo.
- El proyecto 2 se presenta exactamente una hora después que el proyecto 3, en un aula que no es la A2.
- El proyecto 3 se presenta en el aula A2.
- El proyecto 5 se presenta a las 19:00, mientras que el proyecto 6 se presenta en la C2 a última hora.

La salida de MiniZinc se debe formatear de modo que se muestre, para cada estudiante, la siguiente información:

"Nombre": "Proyecto", "Aula", "Hora"

Por poner un ejemplo cualquiera (no necesariamente una solución real del ejercicio):

"Bruno": "A cuántico", "C1", "16:00"*

¿Cuántas soluciones tiene este ejercicio? Si son menos de 15, deben incluirse explícitamente todas las soluciones en la memoria.

El fichero entregado se debe denominar Ejercicio2.mzn.



3. Problema de la mochila multiobjetivo con optimización lexicográfica (1 pto)

Un equipo de intervención técnica debe preparar una mochila con material especializado. Cada objeto tiene asociado un peso, un coste y una utilidad. La mochila está sujeta a limitaciones físicas y económicas, y además deben cumplirse varias restricciones entre objetos. Se desea seleccionar un subconjunto de objetos cumpliendo todas las restricciones y aplicando el siguiente criterio de optimización lexicográfica: primero, maximizar la utilidad total; entre todas las soluciones de utilidad máxima, minimizar el peso total; y, entre todas las soluciones anteriores, minimizar el coste total. Los objetos disponibles son los siguientes:

Objeto	Peso	Coste	Utilidad
GPS	3	8	10
Cámara	2	6	9
Ordenador	4	10	14
Batería extra	2	4	7
Sensor térmico	5	12	16
Botiquín	3	5	8
Radio	4	7	11
Panel solar	6	9	13

Se debe cumplir con todas las restricciones siguientes:

- Peso máximo: la suma de pesos no puede superar 15.
- Dependencia técnica: si se selecciona el "Ordenador", también debe seleccionarse la "Batería extra".
- Compatibilidad energética: si se selecciona el "Sensor térmico", también debe seleccionarse el "Panel solar".
- Seguridad mínima: debe seleccionarse obligatoriamente el "Botiquín".

Es necesario diseñar una función ponderada que convierta este criterio lexicográfico (primero, maximizar utilidad; segundo, minimizar peso; tercero, minimizar coste) en una única función objetivo a maximizar como parte de un COP.

Se tienen que mostrar los objetos seleccionados y los valores finales de utilidad, peso y coste siguiendo el siguiente formato:

```
Objetos seleccionados:  
- Objeto (peso=XX, coste=XX, utilidad=XX)  
- ...
```

```
Utilidad total = YY  
Peso total = ZZ  
Coste total = WW
```

Junto con todo lo anterior, se debe indicar cuántas soluciones óptimas existen y, si son menos de 10, se deben listar explícitamente en la memoria.

El fichero entregado se debe denominar Ejercicio3.mzn.



4. Problema del cuadrado mágico (1.5 pts)

Dado un entero positivo N , se desea construir una matriz cuadrada de tamaño $N \times N$ con números enteros positivos tal que: al menos la mitad de todos los valores de la matriz sean distintos (por ejemplo, si la matriz es 3×3 debe contener al menos 5 números diferentes); si N es par, todos sus valores deben ser pares; si N es impar, todos los valores deben ser impares; y la suma de los elementos de cada fila y de cada columna, y la suma de los elementos de la diagonal principal y de la diagonal secundaria, proporcionen el mismo valor. Entre todas las soluciones posibles, se quiere minimizar la suma total de los elementos de la matriz. El valor máximo permitido en cada celda es 100.

Se debe implementar dicho problema de razonamiento con restricciones (Ejercicio4.mzn) y completar la siguiente tabla:

N	Núm. Valores Distintos	Suma Común	Suma Total Minimizada	Runtime
3				
4				
5				
6				
7				

En caso de que el tiempo (*runtime*) pase de 5 minutos, puede detenerse la ejecución e indicar en la tabla “-” en “Núm. Valores Distintos”, “Suma Común” y “Suma Total”, e ∞ en *runtime*. Es importante remarcar que se deben romper simetrías si estas aparecen², dado que, de lo contrario, estaremos realizando un proceso de búsqueda innecesariamente complejo/extenso.

En la memoria, aparte de cubrir la tabla, se debe crear una gráfica que muestre en el eje de abscisas los valores de N y en el eje de ordenadas el runtime. ¿Qué conclusión se puede extraer del comportamiento observado? ¿Se trata de un problema escalable? Reflexione sobre la clase de complejidad computacional del problema en cuestión, y relaciónelo con los resultados obtenidos.

La ejecución del código debe proporcionar una salida formateada exactamente del siguiente modo (se muestra un ejemplo):

```
N = 3
MAXV = 100
Minimo de valores distintos exigido = 5
Numero de valores distintos = 5
Suma comun de filas/columnas/diagonales = 9
Suma total minimizada = 27
```

```
Matriz:
2 5 2
3 3 3
4 1 4
```

² Es decir, se deben descartar posibles soluciones que representen esencialmente lo mismo (en el Ejercicio 6 se aporta más información al respecto). La idea no es explorar todas las matrices posibles, sino conservar una representante de cada familia de matrices equivalentes por simetría. Por ejemplo, si tenemos esta matriz:

```
9 3 1
5 7 8
2 4 6
```

y otra que es la misma rotada o reflejada, ambas representan esencialmente la misma solución "geométrica". Son simétricas y no es necesario explorarlas todas, dado que, en este ejercicio, no son soluciones realmente diferentes.



5. Problema de planificación de tareas (2 pts)

- a. El legendario DeLorean de Regreso al Futuro debe ser ensamblado lo más rápido posible para poder viajar al pasado. Para ello, contamos con cuatro personas: Marty McFly, Doc Brown, Biff Tannen y George McFly. Marty y Doc son los dos trabajadores principales, y cada tarea debe ser realizada por exactamente uno de ellos. Ninguno de los dos puede realizar dos tareas simultáneamente, aunque sí pueden trabajar en paralelo en tareas distintas. La siguiente tabla muestra las tareas necesarias para construir el DeLorean, junto con la duración de cada una dependiendo de quién la realice, y las tareas que deben completarse antes de que otra pueda comenzar:

Tarea	Duración en días (Marty, Doc)	Predecesoras
A – Montar chasis	10, 16	Ninguna
B – Instalar ruedas	1, 2	A
C – Cableado eléctrico	5, 4	B
D – Motor de fusión nuclear	15, 5	A, C
E – Tablero de control	12, 4	C
F – Instalación y configuración del condensador de flujo (<i>flux capacitor</i>)	13, 5	C, D, E
G – Ajuste aerodinámico	7, 6	C, D
H – Instalación de puertas de ala de gaviota	4, 3	E, G
I – Panel de tiempo	2, 2	F, H
J – Implementación y validación de algoritmos de búsqueda heurística para optimización de caminos	4, 2	G, I
K – Prueba de plutonio	4, 3	H, I

Queremos encontrar una asignación óptima de tareas y tiempos de inicio para completar la construcción en el menor tiempo posible. Se asume que la tarea A comienza en el instante de tiempo 1 y que, si A dura 10 días y empieza el día 1, entonces B puede empezar el día 11.

Además, Biff podrá ser asignado como trabajador de apoyo a Marty o a Doc en determinadas tareas, sujeto a las siguientes condiciones:

1. Biff solo puede apoyar en tareas cuya duración, para el trabajador asignado, sea al menos de 3 días.
2. Biff puede apoyar como máximo en 4 tareas en total.
3. Biff no puede apoyar en tareas que se ejecuten simultáneamente.
4. Biff no puede ayudar en las tareas D ni F, por tratarse de tareas demasiado delicadas.
5. Biff no puede apoyar en más de 2 tareas asignadas a Marty, ni en más de 2 tareas asignadas a Doc.
6. Gracias al apoyo de Biff, la duración de la tarea se reduce: en 3 días si la tarea es A; en 2 días si la tarea es H; en 1 día para cualquier otra tarea elegible.

Por su parte, George McFly también podrá actuar como trabajador de apoyo, pero con un rol más limitado:

1. George solo puede ayudar a Marty, nunca a Doc.
2. Si George ayuda en una tarea, la duración de dicha tarea se reduce en 1 día.



3. George solo puede ayudar en tareas asignadas a Marty cuya duración original para Marty sea de al menos 2 días.
4. George no puede realizar dos apoyos simultáneamente.
5. George no puede coincidir con Biff en una misma tarea.

Por ejemplo:

- si la tarea “A – Montar chasis” es asignada a Marty y recibe apoyo de Biff, su duración pasa de 10 a 7 días;
- si la tarea “E – Tablero de control” es asignada a Marty y recibe apoyo de George, su duración pasa de 12 a 11 días.

En la memoria, se debe indicar con claridad cuál es la duración mínima de la construcción del DeLorean, y se debe incluir un diagrama de Gantt con la planificación encontrada (véase https://en.wikipedia.org/wiki/Gantt_chart). Para este apartado, se debe entregar un fichero denominado Ejercicio5a.mzn, que imprima siguiendo el siguiente formato:

"Letra - DescripciónTarea" inicio día "X" fin día "Y". Asignada a "NombreTrabajador" con apoyo de Biff

Por ejemplo, podríamos tener "A – Montar chasis" inicio día "1" fin día "7". Asignada a "Marty" con apoyo de Biff, lo que significaría que la tarea “Montar chasis” comienza el día 1 y termina el 7 (incluido), y la llevan a cabo Marty y Biff. Esta tarea a Marty le llevaría 10 días pero, como le ayuda Biff, le lleva 3 días menos.

- b. Una vez haya resuelto este problema de optimización de restricciones (COP), plantéelo como un problema de satisfacción de restricciones (CSP) siguiendo las indicaciones de la presentación de la práctica, y calcule el número de soluciones existentes con ese valor mínimo. Es decir, ¿cuántas configuraciones/planes diferentes existen con esa duración mínima? Se debe entregar un fichero denominado Ejercicio5b.mzn, que resuelva el CSP e imprima todos los planes de tiempo mínimo.



6. Problema de conformación de tribunales de defensa de TFG (1.5 pts)

Se acerca Julio y, por tanto, el momento en que muchos estudiantes defenderán sus Trabajos de Fin de Grado (TFGs). Para ello es necesario configurar los tribunales de evaluación, teniendo en cuenta que hay un total de 12 TFGs a evaluar en esta convocatoria, se van a configurar 3 tribunales de defensa y evaluación de TFG. Cada tribunal evaluador consta de 3 personas, cada una de un departamento diferente (DECSAI, LSI e ICAR), y cada tribunal puede evaluar entre 4 y 6 TFGs (ambos inclusive). Disponemos de 10 profesores (profesores 1-4 de DECSAI, profesores 5-7 de LSI, y profesores 8-10 de ICAR). Las defensas de TFG tendrán lugar en una semana concreta (determinada por el calendario de evaluación) y pueden ser cualquier día de la semana, de Lunes a Viernes, en sesión de mañana o tarde, siempre y cuando a los propios profesores les venga bien esa fecha. Como es lógico, un TFG solo es evaluado por un único tribunal. Todo esto que, a priori, suena muy sencillo, en realidad es mucho más complicado de lo que parece:

- Primero, porque cada profesor tiene una apretada agenda (con obligaciones docentes, investigadoras y de gestión) que le impide estar disponible todos los días. En este sentido,
 - o El profesor 1 solo podría participar en el tribunal el martes.
 - o El profesor 2 solo puede lunes, miércoles y viernes.
 - o El profesor 3 solo puede lunes y miércoles.
 - o El profesor 4 solo miércoles y jueves.
 - o El profesor 5 solo lunes y jueves.
 - o El profesor 6 es el único que está disponible todos los días.
 - o El profesor 7 solo viernes.
 - o El profesor 8 solo los miércoles y jueves.
 - o El profesor 9 solo está disponible los jueves.
 - o El profesor 10 solo puede el lunes, martes y viernes.
 - o Los profesores 1, 5 y 8 solo pueden participar en sesión de mañana, mientras que los profesores 2, 4 y 6 solo pueden participar de tarde.
- En segundo lugar, existe la regla de que un profesor que haya supervisado/dirigido un TFG no puede formar parte del tribunal que lo evalúe. En este caso:
 - o El profesor 2 ha dirigido los TFGs 1 y 10.
 - o El profesor 3 ha dirigido el TFG 8, y ha codirigido el TFG 7 con el profesor 8.
 - o El profesor 4 ha dirigido el TFG 6 y 12, y ha codirigido el TFG 4 con el profesor 8.
 - o El profesor 5 ha dirigido el TFG 2.
 - o El profesor 6 ha codirigido con el profesor 7 el TFG 3 y el 5.
 - o El profesor 10 ha codirigido con el profesor 1 el TFG 9 y el 11.

Esto significa que, por ejemplo, el profesor 2 no puede formar parte del tribunal que evalúe el TFG 10.

- Y, junto con lo anterior, existen otra serie de restricciones, acordadas por la Escuela y los departamentos, con el objetivo de que la evaluación de TFGs sea lo más justa y llevadera para todas las partes:
 - o El profesor 10 dispone de más tiempo este cuatrimestre y, como consecuencia, se ha ofrecido voluntario a aparecer, si es necesario, en más de un tribunal (2 como máximo). El resto de profesores solo pueden aparecer en un único tribunal en esta convocatoria.
 - o La temática de los TFGs 6 y 12, ambos dirigidos por el profesor 4, tienen una temática muy similar por lo que parece adecuado que sea el mismo tribunal el que los evalúe. Lo mismo ocurre con los TFGs 1 y 10, dirigidos por el profesor 2. Los otros TFGs son



suficientemente diversos a nivel temático como para no forzar que sean evaluados por el mismo tribunal.

- Un día determinado puede haber más de un tribunal evaluador, pero siempre y cuando no esté implicado un mismo profesor. Es decir, un mismo profesor no puede tener dos tribunales el mismo día (aunque sea en sesiones distintas: mañana y tarde).

Responda en la memoria a las siguientes preguntas de forma razonada:

- ¿Cuál es el número de soluciones válidas obtenidas? En caso de ser 10 soluciones factibles o menos, lístelas explícitamente.
- Discuta la posible existencia de soluciones simétricas. Por soluciones simétricas se entienden aquellas que tienen valores distintos para las variables de la codificación CSP (por lo que MiniZinc las interpreta como soluciones diferentes), pero semánticamente representan la misma solución³. En caso de que la codificación propuesta por el estudiante contenga soluciones simétricas, ¿cómo se podrían evitar y cuál es el número de soluciones (no simétricas) obtenido? Explique cómo se consigue la rotura de simetrías⁴ (variables y/o restricciones utilizadas para ello), y entregue la solución MZN sin simetrías. En caso de que la solución inicial directamente no contuviese simetrías, analice de qué modo sí podría haberlas (es decir, qué podría provocar la existencia de simetrías en este problema) y de qué forma la codificación escogida implícitamente las elimina.

Para este apartado, se debe entregar un fichero denominado Ejercicio6.mzn, que imprima la información siguiendo el siguiente formato:

```
Tribunal 1: "numero profesor"("depto"), "numero profesor"("depto"), "numero  
profesor"("depto") (Día: "dia de la semana", Sesion: "Mañana o tarde")  
TFGs a evaluar: "numeros de TFG separados por comas"
```

```
Tribunal 2: "numero profesor"("depto"), "numero profesor"("depto"), "numero  
profesor"("depto") (Día: "dia de la semana", Sesion: "Mañana o tarde")  
TFGs a evaluar: "numeros de TFG separados por comas"
```

³ Por ejemplo, si tuviésemos Solución1 (Tribunal 1: {Prof1, Prof2, Prof3} a evaluar TFGs 1-5, Tribunal 2: {Prof4, Prof5, Prof6} a evaluar TFGs 6-10) y Solución2 (Tribunal 1: {Prof3, Prof2, Prof1} a evaluar TFGs 1-5, Tribunal 2: {Prof4, Prof5, Prof6} a evaluar TFGs 6-10). En realidad, representarían la misma solución, y solo supondrían una permutación del orden de los profesores del tribunal 1. En este caso, MiniZinc estaría considerando como soluciones diferentes dos soluciones semánticamente idénticas. Esto suele ser resultado de una codificación inadecuada por nuestra parte. Si tenemos esto en cuenta, podremos entender por qué es interesante romper simetrías: porque nos transmiten la impresión de que hay más soluciones de las que realmente hay.

⁴ <https://docs.minizinc.dev/en/stable/efficient.html#symmetry>



7. Planificación de horarios de juego en una semana de la NBA (1.5 ptos)

Se desea planificar el calendario de partidos de una semana de competición para una liga reducida inspirada en la NBA. Participan los siguientes 10 equipos: Celtics, Knicks, Sixers, Heat, Bucks, Lakers, Warriors, Suns, Nuggets y Mavericks. La semana consta de 7 días, numerados del 1 al 7, y cada día dispone de hasta 3 franjas horarias: Tarde, Noche y Prime Time. La liga ya ha decidido qué 12 partidos concretos deben jugarse esa semana. El problema consiste en decidir, para cada uno de esos partidos, en qué día y en qué franja horaria se disputa. Los 12 partidos a programar son los siguientes:

- Celtics vs Knicks
- Heat vs Sixers
- Bucks vs Celtics
- Lakers vs Warriors
- Suns vs Nuggets
- Mavericks vs Lakers
- Warriors vs Suns
- Knicks vs Heat
- Sixers vs Bucks
- Nuggets vs Mavericks
- Celtics vs Lakers
- Heat vs Warriors

En nuestro caso, asumimos que en un partido “X vs Y”, el equipo X juega como local.

Deben cumplirse las siguientes condiciones:

- Cada partido debe jugarse exactamente una vez, en un único día y una única franja.
- Un equipo no puede jugar más de un partido el mismo día.
- En una misma franja horaria puede haber como mucho 2 partidos.
- En un mismo día puede haber como mucho 4 partidos.
- El día 4 no puede haber partidos, dado que corresponde a un festivo nacional.
- El día 7 (domingo) no puede haber partidos en Prime Time (porque ese mismo horario está ocupado con otro evento deportivo multitudinario).
- Los equipos Lakers, Celtics y Warriors no pueden jugar en la franja de Tarde, por razones televisivas.
- Al menos 3 partidos de la semana deben disputarse en Prime Time.
- El partido Lakers vs Warriors debe jugarse el día 2 en Prime Time.
- El partido Celtics vs Lakers debe jugarse el día 6 en Prime Time.
- El partido Bucks vs Celtics debe jugarse también en Prime Time, dado que la liga le va a hacer un homenaje a Ray Allen (jugador importante para ambas franquicias).
- El pabellón de los Knicks no está disponible el día 5 (por lo que Knicks vs Heat no puede jugarse ese día), el pabellón de los Suns no está disponible el día 2 (por lo que Suns vs Nuggets no puede jugarse ese día), y lo mismo ocurre con los partidos Bucks vs Celtics y Nuggets vs Mavericks, que no pueden jugarse en los días 1 y 6, respectivamente.
- En jerga de la NBA, se dice que un equipo tiene un *back-to-back* si juega en dos días consecutivos. A nivel individual, cada equipo puede tener como máximo un *back-to-back* durante la semana. A nivel global, entre todos los equipos, como los *back-to-back* suelen resultar agotadores para los jugadores, puede haber como máximo dos *back-to-back* en toda la semana. Además, los Celtics y los Warriors no pueden tener ningún *back-to-back* esta semana, porque ya han disputado uno recientemente.
- Los días 5 y 6 deben tener al menos 3 partidos cada uno.
- Si un equipo juega en Prime Time, no puede jugar al día siguiente en la franja de Tarde.



- El número de partidos de Tarde debe ser mayor que el número de partidos de Noche; y el número de partidos de Noche debe ser mayor que el número de partidos de Prime Time.

Se pide construir un modelo MiniZinc que encuentre un calendario que satisfaga todas las restricciones anteriores. El fichero a entregar se debe denominar Ejercicio7.mzn. Junto con lo anterior, se debe indicar el número de soluciones encontradas (sin presentar simetrías), y se debe mostrar la primera solución encontrada, formateada exactamente como se muestra a continuación con un ejemplo:

CALENDARIO SEMANAL NBA (CSP)

=====
Back-to-backs totales = 2

Sesion	Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	
Tarde	-	-	MIA-PHI	-	PHX-DEN	NYK-MIA / PHI-MIL	DEN-DAL	
Noche	BOS-NYK	-	-	-	DAL-LAL / MIA-GSW	-	GSW-PHX	
Prime Time	-	LAL-GSW	MIL-BOS	-	-	BOS-LAL	-	

Leyenda:

BOS=Celtics, NYK=Knicks, PHI=Sixers, MIA=Heat, MIL=Bucks,
LAL=Lakers, GSW=Warriors, PHX=Suns, DEN=Nuggets, DAL=Mavericks